

Regularization in Machine Learning

Lecture 2. Regularization in Practice & Deep Learning

Sonnet 5 High

Claude Cowork

August 6, 2026

- Closing the elastic net loop
- Early stopping & dropout
- Data augmentation & batch normalization
- Synthesis & wrap-up

Recap: three lenses on regularization

- **Bias-variance:** a small increase in bias buys a larger drop in variance
- **Geometric:** coefficients are constrained to a norm ball — circle (L2) or diamond (L1)
- **Bayesian:** the penalty encodes a prior belief that coefficients should be small

Today we extend this toolkit — first by closing a loop from last time, then into deep learning.

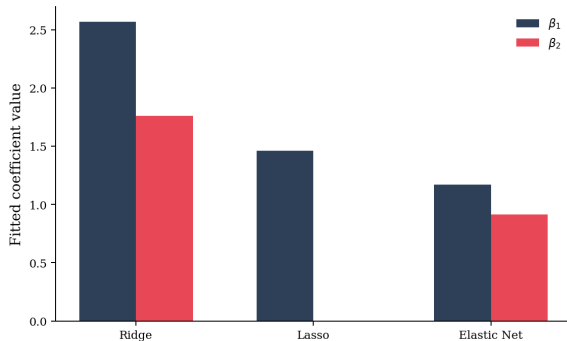
Elastic net: the objective

$$\hat{\beta}_{\text{enet}} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$$

- $\lambda_2 = 0$: recovers lasso exactly
- $\lambda_1 = 0$: recovers ridge exactly
- In between: a tunable blend of both penalties

Elastic net: the grouping effect

- Two features, strongly correlated
- Ridge shrinks them together, roughly equally
- Lasso arbitrarily zeros one out
- Elastic net keeps both nonzero, shrunk together — the best of both



Elastic net: practical guidance

- Default choice when features are correlated *and* you suspect some are irrelevant
- Rarely strictly worse than ridge or lasso alone
- Cost: one extra hyperparameter — the blend between λ_1 and λ_2 — tuned via cross-validation

Regularizing deep networks: why it's different

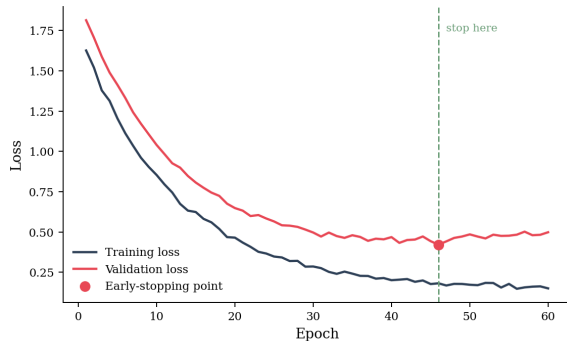
- Deep networks are typically massively overparameterized — far more weights than training examples
- **Weight decay** is the deep-learning name for L2 regularization, applied inside the optimizer instead of in closed form:

$$w \leftarrow w - \eta(\nabla L(w) + \lambda w) = (1 - \eta\lambda) w - \eta \nabla L(w)$$

- Every step shrinks w by $(1 - \eta\lambda)$ before the gradient update — the same ridge penalty from Lecture 1, now applied step-by-step rather than solved in closed form (exact for plain gradient descent; adaptive optimizers like Adam decouple this)
- Still often not enough alone — motivates techniques built around *how* networks are trained: early stopping, dropout, data augmentation, batch normalization

Early stopping: the idea

- Training loss keeps falling with more epochs
- Validation loss eventually turns around and rises
- Early stopping: halt training at the validation minimum, even though training loss would keep falling

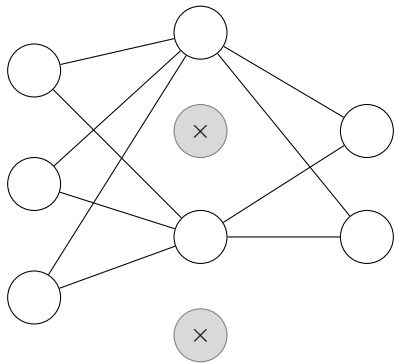


Early stopping: why it's a regularizer

- Weights start near initialization and move further away as training progresses
- Stopping early caps how far the weights can travel
- Informally, this bounds the effective size of the weights — similar in spirit to an explicit penalty, without adding one

Dropout: the mechanism

- At each training step, randomly zero out a fraction of hidden units
- A different random subset is dropped every step
- Remaining units and their connections train normally



Dropout: why it works

- Prevents co-adaptation: no unit can rely on any specific other unit always being present
- Approximately equivalent to training and averaging an ensemble of many thinned subnetworks that share weights
- Both effects push the network toward more robust, redundant representations

Dropout: train vs. test time

During training, each unit is kept with probability p and rescaled:

$$\tilde{h} = \frac{1}{p} \cdot m \odot h, \quad m_i \sim \text{Bernoulli}(p)$$

- Rescaling by $1/p$ keeps the expected activation magnitude unchanged
- At test time: dropout is turned off entirely, no scaling needed — this is “inverted dropout,” the modern standard

Data augmentation: the idea

Same label throughout: "cat"

Original



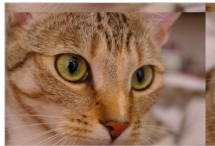
Rotated



Flipped



Shifted



Brightness-jittered



Every transformation above is **label-preserving**: whatever the correct answer was, it still is.

Data augmentation: as regularization

- Augmentation doesn't add new information about the world
- It teaches the model which variations *shouldn't* matter for the task
- This shrinks the effective hypothesis space toward functions invariant to those transformations
- Conceptually similar to constraining the model — just enforced through the data instead of the loss

Batch normalization: the mechanism

For each mini-batch, normalize activations using the batch's own statistics, then apply a learned scale and shift:

$$\hat{x} = \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

- $\mu_{\text{batch}}, \sigma_{\text{batch}}^2$: mean and variance computed from the current mini-batch
- γ, β : learned parameters — the network can undo the normalization if that's optimal

Batch normalization: as implicit regularizer

- Batch statistics are computed from a randomly sampled mini-batch, not the full dataset
- Each example is normalized slightly differently depending on which batch it lands in
- This injects a small amount of noise into training — similar in spirit to dropout
- Primary purpose is optimization stability; the regularizing effect is a side benefit

The full toolkit: a comparison table

Technique	Constrains / perturbs	Extra hyperparameter	Changes training loop?
Ridge	coefficient L2 norm	λ	No
Lasso	coefficient L1 norm	λ	No
Elastic net	both norms	λ_1, λ_2	No
Early stopping	training duration	patience	Yes
Dropout	hidden unit presence	keep prob. p	Yes
Data augmentation	input distribution	transform strength	Yes
Batch norm	activation statistics	(usually none)*	Yes

* γ, β are learned via backprop like ordinary network weights — not tuned hyperparameters.

Choosing in practice

- These techniques are not mutually exclusive — a typical modern setup combines several at once
- Start with weight decay and early stopping: low-cost defaults
- Add dropout for large fully-connected layers
- Add augmentation whenever label-preserving transformations exist for the data type
- Always validate strength/rate choices via cross-validation or a held-out set

Summary & closing

- Elastic net closes the classical toolkit: a tunable blend of L1 and L2
- Early stopping, dropout, data augmentation, and batch norm regularize deep networks in different ways — by capping training time, perturbing units, perturbing inputs, or perturbing activation statistics
- Every technique here trades a little bias for less variance — just through a different mechanism

Now let's put this whole toolkit to work in the lab.